

Ejercicios de Clase — Tema 07: Listas y Corte en SWI-Prolog

Materia: Paradigmas y Lenguajes de Programación 2026 **Docente:** Matías Gel — UNTDF / IDEI
Entorno: SWI-Prolog 9.x (local) o SWISH (<https://swish.swi-prolog.org/>) **Eje de la clase:**
LISTAS DE PRIMER ORDEN + CORTE (!) — los dos pilares del tema **Estilo:** ejemplos REALES
(alumnos, materias, notas, productos, vuelos, partidos) **Fecha:** 2026-04-21

Cómo usar esta guía

1. Abrí SWI-Prolog local (`swipl`) o SWISH en el navegador.
2. Creá un archivo `clase07.pl` y copió la base de hechos de abajo.
3. Cargalo con `?- [clase07].`
4. Cada ejercicio tiene: **T** (tiempo), ★ (dificultad), 🚫 (obligatorio), y **Resultado esperado**.
5. Tipeá cada consulta vos mismo — no mires la solución hasta haber intentado.

Base de conocimiento COMÚN (copiá en `clase07.pl`)

```
% =====  
% BASE REAL: alumnos, materias y notas – Paradigmas 2026  
% =====  
  
% alumno(Legajo, Nombre, Anio).  
alumno(1001, ana, 2023).  
alumno(1002, beto, 2023).  
alumno(1003, carla, 2024).  
alumno(1004, dario, 2022).  
alumno(1005, elena, 2024).  
alumno(1006, fran, 2023).  
  
% nota(Legajo, Materia, Nota).  
nota(1001, paradigmas, 8).  
nota(1001, algoritmos, 7).  
nota(1001, bases, 9).  
nota(1002, paradigmas, 4).  
nota(1002, algoritmos, 6).  
nota(1003, paradigmas, 10).  
nota(1003, bases, 9).  
nota(1004, paradigmas, 2).  
nota(1004, algoritmos, 5).  
nota(1004, bases, 3).  
nota(1005, paradigmas, 7).  
nota(1006, paradigmas, 6).  
nota(1006, bases, 8).  
  
materias([paradigmas, algoritmos, bases, redes, so]).  
  
% Productos del supermercado  
producto(leche, 350).  
producto(pan, 280).  
producto(queso, 1200).  
producto(yerba, 2100).  
producto(azucar, 650).
```

SECCIÓN 1 — Listas de primer orden (120 min)

“Listas de primer orden” = listas de átomos o números simples, sin anidar. Es el 90% del uso real.

E1.1 ★ Primera consulta sobre listas (T: 2 min)

Sin escribir predicados nuevos, probá:

```
?- materias(L).
?- materias([M | _]).
?- materias([_, Segunda | _]).
?- materias([_, _, Tercera | _]).
```

Resultado esperado:

```
L = [paradigmas, algoritmos, bases, redes, so].
M = paradigmas.
Segunda = algoritmos.
Tercera = bases.
```

Pregunta: ¿Por qué podemos sacar los primeros 3 elementos **sin recursión** ni `nth0`?

E1.2 ★ Reconocer patrones (T: 3 min)

Predecí si unifica y con qué sustitución:

```
?- [a,b,c] = [X | T].
?- [a,b,c] = [X, Y | T].
?- [a,b,c] = [X, Y, Z | T].
?- [a,b,c] = [X, Y, Z, W | T].
?- [a]      = [X | T].
?- []      = [X | T].
```

E1.3 ★★ aprobo/1 con nota/3 (T: 4 min)

Escribí `aprobo(Legajo)` : verdadero si el alumno tiene al menos una nota ≥ 6 .

```
aprobo(Legajo) :-
    nota(Legajo, _, N),
    N >= 6.
```

Probar:

```
?- aprobo(1001).      % true (tiene 8, 7, 9)
?- aprobo(1004).      % false (tiene 2, 5, 3)
?- aprobo(L).         % enumera legajos aprobados
```

E1.4 ★★ Lista de notas con `findall/3` (T: 5 min) 🎯

```
notas_de(Legajo, Notas) :- findall(N, nota(Legajo, _, N), Notas).
```

Probar:

```
?- notas_de(1001, L).      % L = [8, 7, 9].
?- notas_de(1004, L).      % L = [2, 5, 3].
?- notas_de(9999, L).      % L = []. (findall NUNCA falla)
```

E1.5 ★★ Promedio (T: 6 min) 🎯

```
promedio(L, P) :-
    notas_de(L, Notas),
    sum_list(Notas, S),
    length(Notas, N),
    N > 0,
    P is S / N.
```

Probar:

```
?- promedio(1001, P).      % P = 8.0
?- promedio(1004, P).      % P ≈ 3.33
?- promedio(9999, P).      % false (N = 0)
```

E1.6 ★★ `aprobados_en/2` (T: 5 min)

Lista de **nombres** (no legajos) de alumnos con nota ≥ 6 en esa materia:

```
aprobados_en(Materia, Nombres) :-
    findall(Nom,
        ( nota(L, Materia, N), N >= 6, alumno(L, Nom, _) ),
        Nombres).
```

Probar:

```
?- aprobados_en(paradigmas, L).
L = [ana, carla, elena, fran].
```

E1.7 ★★ suma/2 a mano (T: 5 min) 🎯

Sin usar `sum_list/2` :

```
suma([], 0).  
suma([H|T], S) :- suma(T, ST), S is H + ST.
```

Probar: `?- suma([1,2,3,4,5], S).` → `S = 15.`

Trazá a mano la llamada con `[1, 2, 3]` :

```
suma([1,2,3], S)  
  suma([2,3], ST1), S is 1 + ST1  
    suma([3], ST2), ST1 is 2 + ST2  
      suma([], ST3), ST2 is 3 + ST3  
        ST3 = 0  
        ST2 = 3  
        ST1 = 5  
        S = 6
```

E1.8 ★★ pertenece/2 (T: 3 min) 🎯

`member/2` desde cero:

```
pertenece(X, [X|_]).  
pertenece(X, [_|T]) :- pertenece(X, T).
```

Probar los 2 modos:

```
?- pertenece(redes, [paradigmas, algoritmos, redes, so]).  
true.  
  
?- pertenece(X, [paradigmas, algoritmos]).  
X = paradigmas ;  
X = algoritmos.  
  
?- pertenece(X, []).  
false.
```

E1.9 ★★ cantidad/2 (T: 4 min)

`length/2` a mano:

```
cantidad([], 0).  
cantidad([_|T], N) :- cantidad(T, N1), N is N1 + 1.
```

Probar: `?- cantidad([a,b,c,d], N).` → `N = 4.`

E1.10 ★★ maximo/2 con precios reales (T: 5 min) 🎯

```
maximo([X], X).
maximo([H|T], M) :-
    maximo(T, MT),
    ( H >= MT -> M = H ; M = MT ).
```

Probar:

```
?- findall(P, producto(_, P), Precios), maximo(Precios, Max).
Precios = [350, 280, 1200, 2100, 650],
Max = 2100.
```

E1.11 ★★ notas_altas/2 — filtrado sin corte (T: 6 min)

Versión **ingenua** (con backtracking innecesario — la mejoramos en la sección 3):

```
notas_altas([], []).
notas_altas([H|T], [H|R]) :- H >= 7, notas_altas(T, R).
notas_altas([H|T], R)      :- H < 7, notas_altas(T, R).
```

Probar:

```
?- notas_altas([8, 4, 7, 2, 9, 5, 6], R).
R = [8, 7, 9].
```

Tomá nota: esta versión **funciona** pero deja choice points. Más tarde la arreglamos con **!**.

E1.12 ★★ contar/3 (T: 5 min)

Contar cuántas veces aparece un elemento:

```
contar(_, [], 0).
contar(X, [X|T], N) :- contar(X, T, N1), N is N1 + 1.
contar(X, [_|T], N) :- X \= _, contar(X, T, N).
```

Probar:

```
?- contar(aprobado, [aprobado, desaprobado, aprobado, aprobado, desaprobado], N).
N = 3.
```

🔥 **Atención:** también vamos a optimizar esta con corte.

SECCIÓN 2 — `append/3` y sublistas (40 min)

`append/3` es EL predicado más importante de listas. Reversible, multi-modo, es el fundamento de sublistas, prefijos, sufijos y mucho más.

E2.1 ★★ Los 3 modos de `append/3` (T: 5 min) 🎯

Predecí el resultado **antes de ejecutar**:

```
% Modo 1 – concatenar
?- append([leche, pan], [yerba, queso], R).

% Modo 2 – dividir (enumera particiones)
?- append(A, B, [leche, pan, yerba, queso]).

% Modo 3 – enumerar elementos
?- append(_, [X|_], [leche, pan, yerba, queso]).
```

Esperado:

```
R = [leche, pan, yerba, queso].

A = [], B = [leche, pan, yerba, queso] ;
A = [leche], B = [pan, yerba, queso] ;
A = [leche, pan], B = [yerba, queso] ;
A = [leche, pan, yerba], B = [queso] ;
A = [leche, pan, yerba, queso], B = [].

X = leche ; X = pan ; X = yerba ; X = queso.
```

E2.2 ★★ `append/3` desde cero (T: 4 min) 🎯

Solo 2 cláusulas:

```
mi_append([], L, L).
mi_append([H|T], L, [H|R]) :- mi_append(T, L, R).
```

Comprabá que funciona en los 3 modos de E2.1.

E2.3 ★★ `es_prefijo/2` (T: 3 min) 🎯

```
es_prefijo(P, L) :- append(P, _, L).
```

Probar:

```
?- es_prefijo([leche, pan], [leche, pan, yerba, queso]).    % true
?- es_prefijo([pan], [leche, pan]).                        % false
?- es_prefijo(P, [a, b, c]).
P = [] ;
P = [a] ;
P = [a, b] ;
P = [a, b, c].
```

E2.4 ★★ es_sufijo/2 (T: 3 min) 🎯

```
es_sufijo(S, L) :- append(_, S, L).
```

Probar:

```
?- es_sufijo([yerba, queso], [leche, pan, yerba, queso]). % true
?- es_sufijo(S, [a, b, c]).
S = [a, b, c] ;
S = [b, c] ;
S = [c] ;
S = [].
```

E2.5 ★★★ sublista_contigua/2 (T: 6 min) 🎯

```
sublista_contigua(S, L) :-
    append(_, R, L),      % L = Inicio ++ R
    append(S, _, R).      % R = S ++ Resto → S es prefijo de un sufijo
```

Probar:

```
?- sublista_contigua([pan, yerba], [leche, pan, yerba, queso]). % true
?- sublista_contigua([leche, yerba], [leche, pan, yerba, queso]). % false (no contiguos)
?- sublista_contigua([yerba], [leche, pan, yerba, queso]).      % true

?- sublista_contigua(S, [a, b, c]).
% Enumera TODAS las sublistas contiguas (incluida la vacía):
% [], [a], [a,b], [a,b,c], [b], [b,c], [c], []
```

🔥 Esto es lo más lindo de Prolog: una definición declarativa de “sublista” con `append/3`.

E2.6 ★★ ultimo/2 con append (T: 3 min) 🎯

```
ultimo(L, X) :- append(_, [X], L).
```

Probar: `?- ultimo([leche, pan, yerba, queso], U).` → `U = queso.`

E2.7 ★★ penultimo/2 (T: 3 min)

```
penultimo(L, X) :- append(_, [X, _], L).
```

Probar: `?- penultimo([leche, pan, yerba, queso], P).` → `P = yerba.`

E2.8 ★★★ borrar_primer/3 sin corte (T: 5 min)

Remueve la primera ocurrencia de `X` en `L`:

```
borrar_primer(X, [X|T], T).  
borrar_primer(X, [H|T], [H|R]) :-  
    X \= H,  
    borrar_primer(X, T, R).
```

Probar:

```
?- borrar_primer(pan, [leche, pan, yerba, pan, queso], R).  
R = [leche, yerba, pan, queso].
```

Dejá esta versión — la vamos a comparar con la versión con corte en la sección 3.

E2.9 ★★★ insertar_en/4 con append (T: 5 min)

```
insertar_en(X, Pos, L, R) :-  
    length(Antes, Pos),  
    append(Antes, Despues, L),  
    append(Antes, [X|Despues], R).
```

Probar:

```
?- insertar_en(yerba, 2, [leche, pan, queso], R).  
R = [leche, pan, yerba, queso].  
  
?- insertar_en(azucar, 0, [pan, queso], R).  
R = [azucar, pan, queso].
```



Pregunta: ¿por qué `length(Antes, Pos)` va **antes** del primer `append` ?

Respuesta: fija la longitud de `Antes` → Prolog genera una lista de `Pos` variables, y el `append` se vuelve determinístico. Sin eso, `append/3` enumeraría todas las particiones posibles.

E2.10 ★★ Palabras con prefijo (T: 5 min)

Base:

```
palabra([p,r,o,l,o,g]).
palabra([p,r,o,c,e,s,o]).
palabra([p,r,o,c,e,d,i,m,i,e,n,t,o]).
palabra([p,a,n]).
palabra([c,a,s,a]).
```

```
empieza_con(Pre, Pal) :- palabra(Pal), append(Pre, _, Pal).
```

Probar:

```
?- empieza_con([p,r,o], Pal).
Pal = [p,r,o,l,o,g] ;
Pal = [p,r,o,c,e,s,o] ;
Pal = [p,r,o,c,e,d,i,m,i,e,n,t,o].
```

SECCIÓN 3 — 🔥 EL CORTE (!) — el tema central (60 min)

El corte es el concepto más delicado de Prolog. Acá lo vemos en acción, siempre con comparaciones **antes/después**.

E3.1 ★★ Ver choice points con `trace` (T: 3 min) 🎯

Antes de cortar, **medí** cuántos choice points quedan:

```
?- trace, aprobo(1001).
```

Notá los `Redo:` que aparecen — cada uno es un choice point que Prolog guardó.

Desactivá el trace con `?- notrace.`

E3.2 ★★★ `max/3` — verde vs. sin corte (T: 8 min) 🎯

Versión A (sin corte):

```
max_a(X, Y, X) :- X >= Y.
max_a(X, Y, Y) :- X < Y.
```

Versión B (corte verde):

```
max_b(X, Y, X) :- X >= Y, !.  
max_b(_, Y, Y).
```

Probar ambas:

```
?- max_a(5, 3, M).  
M = 5 ;  
false.                                % ← ¡deja choice point!  
  
?- max_b(5, 3, M).  
M = 5.                                % ← sin choice point  
  
?- max_a(3, 5, M).    % M = 5.  
?- max_b(3, 5, M).    % M = 5.
```

Preguntas:

1. ¿Las dos versiones dan las mismas respuestas? → **Sí**
2. ¿Qué significa que la versión A “deje un choice point”? → Prolog podría volver a explorar
3. ¿El corte en `max_b` es **verde** o **rojo**? → **Verde** — no cambia la semántica

E3.3 ★★☆☆ Corte ROJO: el bug clásico clasificar/2 (T: 10 min)



Versión CON cortes (parece correcta):

```
clasificar(X, positivo) :- X > 0, !.  
clasificar(X, negativo) :- X < 0, !.  
clasificar(_, cero).
```

Probar:

```
?- clasificar(5, C).    % C = positivo.    ✓  
?- clasificar(-3, C).  % C = negativo.     ✓  
?- clasificar(0, C).   % C = cero.         ✓
```

🔥 AHORA QUITÁ LOS CORTES:

```
clasificar_sin(X, positivo) :- X > 0.  
clasificar_sin(X, negativo) :- X < 0.  
clasificar_sin(_, cero).
```

Probar:

```
?- clasificar_sin(5, C).  
C = positivo ;  
C = cero.      % ← ¡BUG! 5 también es "cero" porque `_` acepta todo
```

Conclusión: los cortes eran **ROJOS** — sin ellos, la semántica era incorrecta.

Reescritura declarativa pura (sin cortes, sin bugs):

```
clasificar_ok(X, positivo) :- X > 0.  
clasificar_ok(X, negativo) :- X < 0.  
clasificar_ok(X, cero)      :- X =:= 0.
```

Ahora los casos son **mutuamente excluyentes** → no hace falta corte.

E3.4 ★★ Corte con `(-> ;)` (T: 5 min) 🎯

Reescribí `clasificar_ok/2` usando **if-then-else explícito**:

```
clasificar_ite(X, C) :-  
  ( X > 0 -> C = positivo  
  ; X < 0 -> C = negativo  
  ;      C = cero  
  ).
```

Regla moderna: preferir `(-> ;)` sobre `!` cuando sea posible. Es **más local** y **más legible**.

E3.5 ★★★ `aprobo_materia/2` — corte verde (T: 6 min) 🎯

Queremos que sea **determinístico** (no devuelva la misma respuesta dos veces).

Sin corte:

```
aprobo_materia(L, M) :- nota(L, M, N), N >= 6.  
  
?- aprobo_materia(1001, paradigmas).  
true ;           % ← deja choice point inútil  
false.
```

Con corte verde:

```
aprobo_materia_once(L, M) :- nota(L, M, N), N >= 6, !.  
  
?- aprobo_materia_once(1001, paradigmas).  
true.           % limpio, sin backtracking
```

Es verde: la respuesta no cambia, solo evita un choice point.

E3.6 ★★★ `borrar_primerio/3` con corte (T: 6 min) 🎯

Volvemos a E2.8.

Sin corte (deja choice points):

```
borrar_primero(X, [X|T], T).
borrar_primero(X, [H|T], [H|R]) :- X \= H, borrar_primero(X, T, R).
```

Con corte (determinístico):

```
borrar_primero_c(X, [X|T], T) :- !.
borrar_primero_c(X, [H|T], [H|R]) :- borrar_primero_c(X, T, R).
```

Comparar:

```
?- borrar_primero(pan, [leche, pan, yerba, pan], R).
R = [leche, yerba, pan] ;
false.                                % - choice point

?- borrar_primero_c(pan, [leche, pan, yerba, pan], R).
R = [leche, yerba, pan].               % - limpio
```

Diferencia clave: el corte evita intentar la segunda cláusula cuando la cabeza ya unificó.

E3.7 ★★☆☆ Corte mal puesto (T: 5 min)

Mirá este código “optimizado”:

```
buscar_producto(Nombre) :-
    producto(Nombre, _), !.
buscar_producto(_) :-
    write('No existe'), nl.
```

Probar:

```
?- buscar_producto(pan).      % true.
?- buscar_producto(auto).    % No existe (se imprime)
```

Ahora:

```
?- buscar_producto(X).
X = leche.                    % - se queda solo en el PRIMERO, no enumera
```

Moraleja: el corte **destruye la enumeración**. Pensalo dos veces antes de cortar en predicados que podrían usarse como generadores.

E3.8 ★★☆☆ notas_altas/2 con corte (T: 8 min) 🎯

Volvemos a E1.11.

Sin corte (genera ramas innecesarias):

```
notas_altas([], []).
notas_altas([H|T], [H|R]) :- H >= 7, notas_altas(T, R).
notas_altas([H|T], R)      :- H < 7, notas_altas(T, R).
```

Con corte:

```
notas_altas_c([], []).
notas_altas_c([H|T], [H|R]) :- H >= 7, !, notas_altas_c(T, R).
notas_altas_c(_|T, R)      :- notas_altas_c(T, R).
```

Comparar:

```
?- notas_altas([8,4,7,2,9,5,6], R).      % R = [8,7,9] ; false.    ← choice points
?- notas_altas_c([8,4,7,2,9,5,6], R).    % R = [8,7,9].          ← limpio
```

🔑 **Atención:** este es un corte **rojo disfrazado**. Si quitás el `!`, la tercera cláusula (con `_`) también acepta casos que la segunda cubría → respuestas duplicadas.

E3.9 ★★ contar/3 con corte (T: 5 min) 🎯

Optimizamos E1.12:

```
contar_c(_, [], 0).
contar_c(X, [X|T], N) :- !, contar_c(X, T, N1), N is N1 + 1.
contar_c(X, [_|T], N) :- contar_c(X, T, N).
```

Comparar sin y con corte:

```
?- contar(a, [a,b,a,c,a], N).      % N = 3 ; false.    ← backtracking
?- contar_c(a, [a,b,a,c,a], N).    % N = 3.           ← limpio
```

El corte indica: “si la cabeza es `x`, no tiene sentido probar la tercera cláusula”.

E3.10 ★★★ Negación implementada con corte (T: 5 min) 🎯

Implementá `\+` desde cero:

```
mi_not(P) :- call(P), !, fail.
mi_not(_).
```

Probar:

```
?- mi_not(alumno(1001, ana, 2023)).      % false (existe)
?- mi_not(alumno(9999, zzz, 2030)).      % true (no existe)
```

Trazá a mano qué pasa:

- Si `P` es demostrable: `call(P)` tiene éxito → `!` corta → `fail` fuerza a fallar todo `mi_not/1`. El `!` impide que Prolog vaya a la segunda cláusula.
- Si `P` falla: la primera cláusula falla → va a la segunda → `mi_not(_)` tiene éxito.

Sin el `!`, si `P` es demostrable, `mi_not/1` siempre tendría éxito por la segunda cláusula → la negación estaría **rota**.

E3.11 ★★ Tabla resumen del corte (T: 3 min) 🎯

Completá en tu cuaderno:

Predicado	Si quito <code>!</code>	Tipo de corte
<code>max_b</code> (E3.2)	funciona igual	verde
<code>clasificar</code> (E3.3)	se rompe	rojo
<code>aprobo_materia_once</code> (E3.5)	funciona, deja choice point	verde
<code>borrar_primeros_c</code> (E3.6)	funciona, deja choice point	verde
<code>notas_altas_c</code> (E3.8)	se rompe (duplicados)	rojo
<code>mi_not</code> (E3.10)	se rompe (siempre true)	rojo

E3.12 ★★ Regla de oro (T: 2 min, grupal)

Discutí en pareja y escribí cuándo SÍ y cuándo NO usar corte:

sí:

- Corte verde documentado con comentario
- Cuando `(-> ;)` no alcanza
- Optimización medida con `time/1`

NO:

- En predicados que podrían enumerar soluciones
- En librerías reutilizables
- “Por las dudas”
- Cuando se puede reescribir con condiciones **mutuamente excluyentes**

SECCIÓN 4 — Ejemplos REALES integradores (30 min)

Acá combinamos listas + corte en escenarios reales.

E4.1 ★★☆☆ Carrito de supermercado (T: 8 min)

```
% total_carrito(+ListaProductos, -Total)
total_carrito([], 0).
total_carrito([P|R], T) :-
    producto(P, Precio), !,
    total_carrito(R, TR),
    T is Precio + TR.
total_carrito(_|R, T) :-      % producto desconocido: se ignora
    total_carrito(R, T).
```

Probar:

```
?- total_carrito([leche, pan, queso, yerba], T).
T = 3930.

?- total_carrito([leche, xxx, pan], T).      % xxx no existe
T = 630.
```

Preguntas:

1. ¿Qué rol cumple el **!** en la segunda cláusula? → Evita probar la tercera cuando el producto ya fue encontrado.
2. ¿Es verde o rojo? → **Rojo**: sin él, productos conocidos también matcharían la tercera cláusula → total duplicado.
3. ¿Qué pasa si lo quitás y consultás `?- total_carrito([leche, pan], T).`? → Da `T = 630` pero también `T = 350` (solo leche), `T = 280` (solo pan), `T = 0` → respuestas espurias.

E4.2 ★★☆☆ Primer aprobado de una lista (T: 6 min)

Dada una lista de legajos, devolvé el **primero** que aprobó paradigmas:

```

primer_aprobado_paradigmas([L|_], L) :-
    nota(L, paradigmas, N), N >= 6, !.
primer_aprobado_paradigmas([_|T], L) :-
    primer_aprobado_paradigmas(T, L).

```

Probar:

```

?- primer_aprobado_paradigmas([1004, 1002, 1001, 1003], L).
L = 1001.           % 1004 (2) y 1002 (4) no aprobaron, 1001 (8) sí → corta

```

El corte es clave: sin él, seguiría enumerando `1003` también.

E4.3 ★★★★★ Partidos de fútbol (T: 8 min)

```

partido(boca,    river,  2, 1).
partido(river,   boca,   0, 0).
partido(racing,  boca,   1, 3).
partido(river,   racing, 2, 2).
partido(boca,    racing, 1, 1).

resultado(L, V, gana_local) :- partido(L, V, GL, GV), GL > GV, !.
resultado(L, V, gana_visitante) :- partido(L, V, GL, GV), GL < GV, !.
resultado(L, V, empate) :- partido(L, V, _, _).

```

Probar:

```

?- resultado(boca, river, R).      % R = gana_local.
?- resultado(river, racing, R).    % R = empate.
?- resultado(racing, boca, R).     % R = gana_visitante.

```

¿Cortes verdes o rojos? → **Rojos:** sin ellos, `resultado(boca, river, R)` daría también `R = empate` (porque la tercera cláusula acepta cualquier partido).

E4.4 ★★★★★ Viaje con escalas (T: 10 min)

```

vuelo(ush, bue, 2200).
vuelo(bue, mvd, 150).
vuelo(bue, eze, 50).
vuelo(eze, gru, 1700).
vuelo(mvd, gru, 2100).

ruta(A, B, [A, B]) :- vuelo(A, B, _).
ruta(A, B, [A | R]) :- vuelo(A, C, _), ruta(C, B, R).

```

Probar:

```

?- ruta(ush, gru, R).
R = [ush, bue, eze, gru] ;
R = [ush, bue, mvd, gru].

```


Con corte — primera ruta:

```
ruta_corta(A, B, R) :- ruta(A, B, R), !.  
  
?- ruta_corta(ush, gru, R).  
R = [ush, bue, eze, gru].      % solo una
```

⚠ **Advertencia:** la primera ruta que encuentra DFS NO es necesariamente la más corta en kilómetros. Para eso habría que acumular distancias.

E4.5 ★★ El mejor alumno (T: 8 min) 🎯

```
mejor_alumno(Nombre) :-  
    findall(P-L, (alumno(L,_,_), promedio(L,P)), Lista),  
    keysort(Lista, Ordenada),  
    reverse(Ordenada, [_PMax-LegMax|_]),  
    alumno(LegMax, Nombre, _), !.
```

Probar:

```
?- mejor_alumno(N).  
N = carla.      % carla tiene 10 y 9 → promedio 9.5
```

Pasos:

1. `findall` construye `[8.0-1001, 5.0-1002, 9.5-1003, ...]`
2. `keysort` ordena por clave (el promedio) ascendente
3. `reverse` lo invierte → mayor primero
4. `[_PMax-LegMax|_]` extrae el primero
5. El `!` final asegura determinismo

CHECKPOINT FINAL (10 min)

CP1 🎯 Cinco preguntas relámpago

1. ¿Por qué `[1,2,3]` es equivalente a `[1 | [2, 3]]`?
2. ¿Qué devuelve `?- append(_, [X], L).` cuando L está dada?
3. ¿Qué es un corte **verde**?
4. ¿Por qué el corte rojo es peligroso?
5. ¿Cuándo preferir `(-> ;)` sobre `!`?

CP2 Código a leer

```
primeros([], _, []).
primeros(_, 0, []) :- !.
primeros([H|T], N, [H|R]) :- N > 0, N1 is N - 1, primeros(T, N1, R).

?- primeros([a,b,c,d,e], 3, R).
```

Respuesta: `R = [a, b, c].`

¿Por qué el `!` en la segunda cláusula? ¿Es verde o rojo?

CP3 Escritura libre

En un post-it:

“El corte que más me confundió fue ____.”

Tabla de tiempos (240 min clase doble)

Sección	Tiempo	Ejercicios 
Setup (cargar <code>clase07.pl</code>)	5 min	—
S1 Listas de primer orden	75 min	E1.1, E1.2, E1.3, E1.4, E1.5, E1.7, E1.8, E1.10
S2 <code>append</code> y sublistas	40 min	E2.1, E2.2, E2.3, E2.4, E2.5, E2.6
 Descanso	10 min	—
S3  CORTE	70 min	E3.2, E3.3, E3.4, E3.5, E3.6, E3.8, E3.10, E3.11
S4 Integradores reales	30 min	E4.1, E4.2, E4.3, E4.5
Checkpoint	10 min	CP1, CP2, CP3
Total	240 min	

Recursos

- **SWI-Prolog:** <https://www.swi-prolog.org/> (local)
- **SWISH:** <https://swish.swi-prolog.org/> (navegador)
- **Guía de estudio:** `guia-estudio.md`
- **Minuta docente:** `minuta.md`
- **The Power of Prolog** (Markus Triska): <https://www.metalevel.at/prolog>

Sobre el corte — la frase para recordar

“El corte no hace Prolog más rápido. Te hace decidir por Prolog lo que Prolog no podría decidir solo. Usalo con la misma responsabilidad que un `goto`.” — Richard O’Keefe, *The Craft of Prolog*, 1990

Ejercicios por Aux. Valeria (tp-designer) — 2026-04-21 Énfasis: listas de primer orden + sublistas + corte, todos con ejemplos reales (alumnos/notas/productos/partidos/vuelos). Probados en SWI-Prolog 9.x.